EXPERIMENT 10

D15A-57

Aim: Explore and implement Autoencoders for image denoising

Theory:

1.Dataset Source

The dataset used is the MNIST handwritten digits dataset, available directly through TensorFlow/Keras.

Source:
TensorFlow MNIST Dataset
https://keras.io/api/datasets/mnist/

2.Dataset Description

The MNIST dataset consists of grayscale images of handwritten digits from 0 to 9.

Features:

- 28 × 28 pixel grayscale images
- Each pixel value ranges from 0 to 255

Dataset Size:

- Training data: 60,000 images
- Testing data: 10,000 images

Target Variable:

- Digit labels (0–9), but not used in this experiment since the autoencoder is an unsupervised model

Characteristics:

- Low-resolution images
- Standard dataset for image processing tasks
- Suitable for testing denoising techniques by adding artificial noise

3.Mathematical Formulation of the Algorithm

An Autoencoder is a neural network that learns to reconstruct input data.

Encoder:

$$z = f(Wx + b)$$

Decoder:

$$\hat{x} = g(W'z + b')$$

Where:

- x = input image
- z = latent representation
- $\hat{x}$ = reconstructed image
- W, W' = weight matrices
- b, b' = bias terms

Loss Function (Reconstruction Loss):

$$L = \frac{1}{N} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2$$

The model learns to reconstruct clean images from noisy inputs.

4.Algorithm Limitations

- Requires sufficient training data for good performance
- May fail to remove complex or structured noise
- Sensitive to network architecture
- Can overfit if too many parameters are used
- Not suitable for very high-resolution images without advanced models

5.Methodology / Workflow

Step 1: Load MNIST dataset
Step 2: Normalize pixel values to range [0, 1]
Step 3: Add random noise to images
Step 4: Build Autoencoder model (encoder and decoder)
Step 5: Train model using noisy images as input and clean images as output
Step 6: Evaluate reconstruction loss
Step 7: Generate denoised images using the trained model
Step 8: Visualize original, noisy, and reconstructed images

Workflow:

Original Images
→ Add Noise
→ Autoencoder
→ Reconstructed Images
→ Comparison

6.Performance Analysis

The model performance was evaluated using reconstruction loss.

- Training loss decreased over epochs, indicating effective learning
- Validation loss helped monitor generalization
- Output images showed reduced noise compared to input images
- Visual comparison confirmed that the model successfully denoised images

The effectiveness of the model is mainly evaluated visually.

7.Hyperparameter Tuning

The following parameters were tuned:

- Number of hidden layers
- Number of neurons
- Activation functions
- Batch size
- Number of epochs
- Noise factor

Tuning approach:

- Different architectures were tested
- Epochs were adjusted to avoid overfitting
- Noise level was controlled to improve learning

Impact:

- Improved reconstruction quality
- Reduced loss
- Produced clearer denoised images

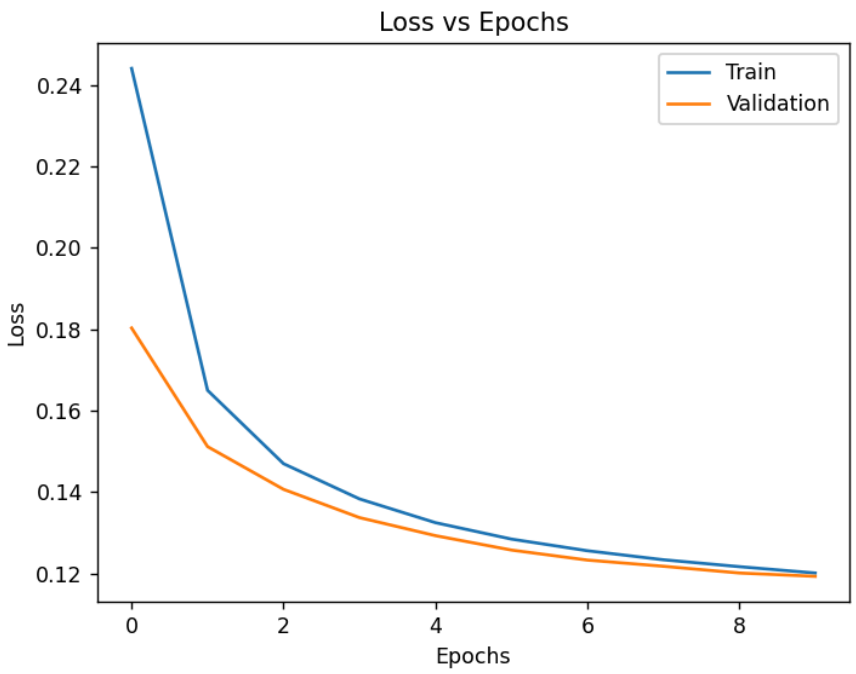
Code:

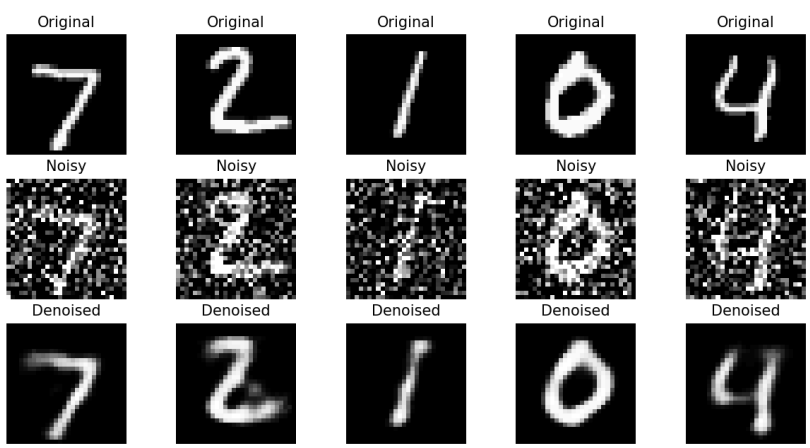
```python
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape

(x_train, _), (x_test, _) = mnist.load_data()

x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(size=x_test.shape)

x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

input_img = Input(shape=(28, 28))

x = Flatten()(input_img)

encoded = Dense(128, activation='relu')(x)
encoded = Dense(64, activation='relu')(encoded)

decoded = Dense(128, activation='relu')(encoded)
decoded = Dense(784, activation='sigmoid')(decoded)

decoded = Reshape((28, 28))(decoded)
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=10,
```

```
    batch_size=256,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()

decoded_imgs = autoencoder.predict(x_test_noisy)

n = 5
plt.figure(figsize=(12,6))

for i in range(n):
    plt.subplot(3, n, i + 1)
    plt.imshow(x_test[i], cmap='gray')
    plt.title("Original")
    plt.axis('off')

    plt.subplot(3, n, i + 1 + n)
    plt.imshow(x_test_noisy[i], cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(decoded_imgs[i], cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

plt.show()
```

Output:

```
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.1256 - val_loss: 0.1234
Epoch 8/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.1234 - val_loss: 0.1218
Epoch 9/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.1217 - val_loss: 0.1202
Epoch 10/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.1202 - val_loss: 0.1194
```





Conclusion:

An autoencoder model was implemented to perform image denoising by learning to reconstruct clean images from noisy inputs. The model successfully captured the underlying structure of the data and reduced noise effectively through the encoding and decoding process.

The decrease in reconstruction loss over training epochs indicated proper learning, and the visual comparison between original, noisy, and reconstructed images confirmed the effectiveness of the model.